Neoepitope selection pipeline (nepiDx) Manual v1.0.0

Yuliu Guo

October 19, 2026

Contents

1 Getting Started **3**
1.1 Download the pipeline 3

2 Preparing inputs **3**
2.1 Required columns 3
2.2 Missing partial inputs 4
2.3 Example input table 4

3 Configuration **6**
3.1 General paths 6
3.2 Gene signature files 6
3.3 Additional files 6
3.4 Parameters 7
3.5 Reference files 7
3.6 Example of config.yaml 8

4 Running the Pipeline **9**
4.1 Example command 9
4.2 Relevant options 9

5 Output **10**
5.1 Directory structure 10
5.2 Output files 11

5.2.1 SNVs annotation and sequence reconstruction 11
5.2.2 Gene fusion annotation and sequence reconstruction . . 13
5.2.3 SVs annotation and sequence reconstruction 14
5.2.4 HLA typing and HLA loss-of-heterozygosity 16
5.2.5 Peptide-MHC binding affinity, stability, and immunogenicity prediction 16
5.2.6 Gene signature scores 21
5.2.7 Additional RNA supporting 21
5.2.8 Tumor ploidy and purity 21
5.2.9 MS-based target-specific peptide 22

6 Importing results into the database 26
6.1 Database scripts 26
6.2 Creating the database 27
6.3 Inserting pipeline results 27
6.4 Database import requirements 27

1 Getting Started

1.1 Download the pipeline

This neoepitope selection pipeline can be downloaded from GitLab: `https://github.com/yuliu-guo/nepiDx`

2 Preparing inputs

This Snakemake workflow requires an input file containing the necessary sample metadata.

2.1 Required columns

At minimum, the input file must include a `patient_id` column.

SNV VCF files are expected to be annotated using VEP and should be provided in the column `snv_somatic_annot_vcf_path`.

RNA-seq STAR-aligned CRAM or BAM files should be provided in the column `RNA_cram_path`. STAR raw count output files, `ReadsPerGene.out.tab`, should be provided in the column `STARcounts_RefSeq_path`.

Tumor WGS/WES alignment files in CRAM or BAM format should be provided in the column `Paired_tumor_DNA_cram_path`. Matched normal DNA alignment files, where available, should be provided in the column `Paired_normal_DNA_cram_path`.

Gene fusion results can be provided from up to three tools. The only required gene fusion input is the Arriba output, which should be provided in the column `gene_fusion_arriba`. Where applicable, gene fusion results from STAR-Fusion and FusionMap can also be provided in the columns `gene_fusion_star` and `gene_fusion_FusionMap`, respectively.

Structural variant (SV) VCF files, either annotated or raw, can be provided in the column `sv_somatic_raw_manta`. Transcriptome expression metrics generated by Salmon can be provided in the column `transcriptome_path`.

The input file can be either comma-separated or tab-separated. The separator must be specified in the `Snakefile`. The path to the input file should

be defined in `config.yaml`, which should be located in the same directory as the `Snakefile`. List columns, such as:

- `patient_id`
- `snv_somatic_annot_vcf_path`
- `STARcounts_RefSeq_path`
- `RNA_cram_path`
- `Paired_normal_DNA_cram_path`
- `Paired_tumor_DNA_cram_path`
- `gene_fusion_arriba`
- `gene_fusion_star`
- `gene_fusion_FusionMap`
- `sv_somatic_raw_manta`
- `transcriptome_path`

2.2 Missing partial inputs

The workflow handles the situation with input files partially missing. When one or more input files are missing for a given patient, the relevant Snakemake rules are selectively skipped or executed based on the available data. More details can be found under workflow/initial_setup.smk.

2.3 Example input table

patient_id	**snv_somatic annot_vcf_path**	**STARcounts RefSeq_path**	**RNA cram_path**	**...**	**gene_fusion arriba**	**...**
patient1	`vep.vcf.gz`	`ReadsPerGene .out.tab`	`RNA.cram`	...	`arriba.tsv`	...
patient2	`vep.vcf.gz`	NA	NA	...	NA	...
patient3	NA	`ReadsPerGene .out.tab`	`RNA.bam`	...	`arriba.tsv`	...

Table 1: Example input table for nepiDx. File names are mocked examples, and missing inputs are indicated by `NA` here but should be left empty.

3 Configuration

The configuration file defines input paths, output locations, reference files, and pipeline parameters. The names of the configuration fields must be kept unchanged.

3.1 General paths

`patient_list` Path to the input file.

`scratch` Path to the scratch directory used for intermediate files. In this workflow, BAM files converted from CRAM files are stored in this directory.

`results_folder` Path to the directory where final results will be saved.

`logs_folder` Path to the directory where log files will be saved.

`singularity_env_folder` Path to the directory where all Singularity containers are stored.

`p_group_file` Path to the P-group file used for reporting ambiguous HLA allele typings. This file can be downloaded from: `https://hla.alleles.org/pages/p&g_groups/p_groups/`

3.2 Gene signature files

`gene_signatures_files` File containing the genes included in each gene signature, together with the corresponding calculation method.

`gene_signatures_PC1_file` Precomputed cohort-specific PC1 file.

`gene_signatures_zscore_file` Precomputed cohort-specific z-score file.

3.3 Additional files

`human_k9` SQLite database containing human wild-type 9-mer peptides constructed using 4-amino-acid seeds. Additional details on how to generate this database can be found under `/workflow/02_mutated_peptide/scripts/build_wt_kmer_db_sqlite.py`.

`purple_ref_dir` Directory containing the reference files required by PURPLE, which is used to calculate tumor purity and ploidy. The reference files can be downloaded from: `https://github.com/hartwigmedical/hmftools/tree/master/purple`

`patient_database_path` Path to the pre-generated database where results can be stored, if database output is desired. Additional details on how to generate this database can be found under `/scripts`.

3.4 Parameters

The following parameters are defined under the `params` section of the configuration file.

`k_mer` Numeric value specifying the peptide length. This value must match the wild-type k-mer database specified by `human_k9`. In this workflow, only 9-mer examples are provided.

`min_alt_base_snv` Numeric value specifying the minimum number of RNA-seq reads supporting the alternative allele for somatic SNVs.

`min_fusion_read_support` Numeric value specifying the minimum number of supporting reads from the STARFusion read-support column.

`max_hamming_similarity` Maximum allowed Hamming similarity. The threshold is exclusive, when this value is set to `1`, only candidates with similarity `< 1` are retained.

`max_blosum62_2_similarity` Maximum allowed similarity based on the BLOSUM62-2 score. The threshold is exclusive, when this value is set to `1`, only candidates with similarity `< 1` are retained.

`hla_quality_threshold` Numeric value specifying the HLA allele quality threshold reported by T1K.

3.5 Reference files

The following reference files are defined under the `reference` section of the configuration file.

`GRCh38_fasta` Path to the GRCh38 human reference genome FASTA file.

`hla_rna_coord` T1K HLA-typing RNA coordinate reference file. This file can be downloaded from: `https://github.com/mourisl/T1K`

`hla_rna_ref` T1K HLA-typing RNA reference file. This file can be downloaded from: `https://github.com/mourisl/T1K`

`hla_dna_coord` T1K HLA-typing DNA coordinate reference file. This file can be downloaded from: `https://github.com/mourisl/T1K`

`hlla_dna_ref` T1K HLA-typing DNA reference file. This file can be downloaded from: `https://github.com/mourisl/T1K`

3.6 Example of config.yaml

```
patient_list: 'input.tsv'
scratch: '/scratch'
results_folder: '/results'
logs_folder: '/logs'
singularity_env_folder: '/singularity_envs'
p_group_file: '/ref/hla_nom_p.txt'

gene_signatures_file: '/ref/gene_signatures_with_method.
   tsv'
gene_signatures_PC1_file: '/ref/PC1_gene_sig_ref.tsv'
gene_signatures_zscore_file: '/ref/z_score_gene_sig_ref.
   tsv'

human_k9: 'human_k9.sqlite'
purple_ref_dir: '/ref/purple'
patient_database_path: 'cohort_database.db'

params:
  K_mer: 9
  min_alt_base_snv: 1
  min_fusion_read_support: 1
  max_hamming_similarity: 1
  max_blosum62_2_similarity: 1
  hla_quality_threshold: 10

reference:
  GRCh38_fasta: 'GRCh38.fasta'
```

```
  hla_rna_coord: '/ref/hlaidx_rna_coord.fa'
  hla_rna_ref: '/ref/hlaidx_rna_seq.fa'
  hla_dna_coord: '/ref/hlaidx_dna_coord.fa'
  hla_dna_ref: '/ref/hlaidx_dna_seq.fa'
```

4 Running the Pipeline

nepiDx is implemented as a modular Snakemake workflow. Each subdirectory contains rules and scripts for a specific analysis component. The prefix of each workflow subdirectory, such as 00_, 01_, indicate the approximate logical order of the analysis instead of the actual execution order. This workflow was built and tested with Snakemake version 7.25.0.

Run the workflow from the `/neoepitope/snakemake_neo/run/` directory. This directory must contain the main workflow directory and the configuration YAML file, as described in 3.6.

The examples below explicitly calling the snakefile by using `--snakefile/-s snakefile`. If the snakefile is named as `Snakefile` or `snakefile` and located in the same working directory as the config file and workflow directory, the `-s` option may be omitted.

4.1 Example command

```
snakemake -s snakefile --profile ../pbs-default/ -j 2 -k -p
```

4.2 Relevant options

Use `--dry-run/-n` to do a dry run. A dry run is recommended before executing the whole workflow, especially after changing the snakefile, configuration, input data, or any Snakemake rules.

Use `--printshellcmds/-p` to print out the shell commands that will be executed.

Use `--cores/-c` to specify the maximum number of CPU cores that Snakemake may allocate concurrently. Increase allowed core usage only when there are has sufficient CPU capacity, memory, temporary storage, and disk space available.

Use `--keep-going/-k` to allow jobs, which are independent of a failed job to continue. Omit this option when the workflow should stop after the first failed job.

Use `--printshellcmds/-p` option prints each shell command before it is executed and is recommended for troubleshooting.

Use `--jobs/-j` to set the maximum number of cluster jobs that may be submitted at the same time. For local Snakemake workflow, -j is an alias for -c.

Optionally, the snakemake workflow can be submitted as cluster jobs, use `--profile` to direct Snakemake find the settings regarding the compute environment. Per-job CPU, memory, and walltime requirements should be defined by the workflow rules or by the PBS profile and mapped to the corresponding PBS scheduler options. If the profile directory is moved, provide either its new relative path or its absolute path.

Use `--latency-wait/-w` when completed output files become visible to the Snakemake controller after a short delay on shared file system.

Use `--rerun-incomplete/--ri` to continue an interrupted run, ore resumed by running the same Snakemake command again.

Do not start a second Snakemake workflow while jobs from the previous workflow remain active. Doing so may submit duplicate jobs or cause multiple processes to write to the same output files.

Snakemake execution logs are stored under:

`.snakemake/log/`

5 Output

5.1 Directory structure

All output files are saved under sample specific subfolders with provided `patient_id` from input file. Within each sample specific directory, outputs are organized into subdirectories corresponding to different analysis steps:

- `00_variants_summary/`

- 01_Hla-typing/
- 02_mutated_peptide/
- 03_pMHC_prediction/
- 04_neoepitopes/
- 05_TME/
- 06_isovar/
- 07_amber/
- 07_cobalt/
- 07_purple/
- 07_spechla/
- 08_immunogenicity/
- 09_diam/

5.2 Output files

5.2.1 SNVs annotation and sequence reconstruction

The 00_variants_summary/*_SNV_summary.csv file contains annotated Single nucleotide variants (SNVs) variants and their associated protein-sequence and RNA-expression information. The output file contains following columns:
ID: unique identifier assigned to the variant, such as s1, s2.
CHROM: chromosome position of the variant.
POS: genomic coordinate of the variant.
REF: reference allele.
ALT: alternative allele.
QSS: Somatic SNV quality score.
SomaticEVS: empirical somatic variant score reported by the variant caller.
FILTER: variant-calling filter status. Variants that pass all applied filters are typically labelled PASS. Variants that has SomaticEVS below threshold is labelled LowEVS. Variants with tumor or normal sample read depth at the locus below 2 is labelled LowDepth.
Consequence: predicted functional consequence of the variant by VEP, such as a missense, inframe_insertion, inframe_deletion, frameshift, ect.

IMPACT: predicted severity of the variant consequence, typically classified as HIGH, MODERATE, LOW, or MODIFIER.
SYMBOL: gene symbol associated with the variant.
refseq: RefSeq transcript or protein identifier used for annotation.
ENSP: Ensembl protein identifier.
HGVSp: protein-level variant description following Human Genome Variation Society nomenclature.
Protein_position: position of the affected amino acid within the protein anootated by VEP.
Amino_acids: reference and alternative amino acids associated with the variant annotated by VEP.
transcript_TPM: expression of the selected transcript in transcripts per million (TPM), available when transcript quant.sf was provided when running the workflow, otherwise left empty.
transcript_NumReads: estimated number of reads assigned to the selected transcript, available when transcript quant.sf was provided when running the workflow, otherwise left empty.
gene_count_TPM: gene-level expression value reported in TPM, available when STAR read counts er gene ReadsPerGene.out.tab file was provided when running the workflow, otherwise left empty.
flanked_mutated_protein_sequence: mutated protein sequence surrounding the affected amino acid.
flanked_mutated_protein_sequence_52aa: 52-amino-acid representation of the mutated protein sequence surrounding the affected position, used in downstream peptide-generation steps.
nmer: list of mutated peptide sequences sliced in to nmer. By defalut is 9-mer.
RNA_ALT_N: number of RNA-sequencing reads supporting the alternative allele, available when RNA-seq BAM file was provided when running the workflow.
RNA_TOTAL_N: total number of RNA-sequencing reads covering the variant position.
RNA_VAF: RNA variant allele frequency, calculated as the proportion of reads supporting the alternative allele among all reads covering the position.

The `02_mutated_peptide/*_snv_similarity.csv` file contains the similarity results for the SNV-derived mutated peptides. The output file contains

the following columns:
ID: unique identifier assigned to the SNV, such as s1, s2.
SYMBOL: gene symbol associated with the SNV.
RNA_ALT_N: number of RNA-sequencing reads supporting the alternative allele, available when an RNA-seq BAM file was provided when running the workflow.
flanked_mutated_protein_sequence: mutated protein sequence surrounding the affected amino acid from which the n-mer peptides were generated.
Mut_peptide: mutated n-mer peptide generated from the flanked mutated protein sequence.
WT_peptide: wildtype peptide used as the reference for comparison with the mutated peptide.
hamming_similarity: similarity score between the mutated and wildtype peptides based on identical amino acids at corresponding positions.
blosum62_2_similarity: similarity score between the mutated and wildtype peptides calculated using the BLOSUM62 amino acid substitution matrix.

5.2.2 Gene fusion annotation and sequence reconstruction

The `00_variants_summary/*_genefusion_summary.csv` file contains gene-fusion events, their predicted fusion-protein sequences, and supporting expression information. The output file contains following columns:
ID: unique identifier assigned to the fusion event, such as g1, g2.
CHROM: chromosome associated with the fusion breakpoint represented in the summary.
POS: genomic position of the represented fusion breakpoint.
REF: retained for the structural consistency purpose, with fixed value fusion.
ALT: retained for the structural consistency purpose, with fixed value fusion.
Protein_position: position of the fusion junction within the predicted fusion protein.
gene_name: name or combined label of the genes involved in the fusion.
variant_type: classification of the fusion event.
fusion_amino_acid_sequence: predicted amino acid sequence of the fusion protein.
transcript_name: transcript identifier or identifiers used to construct the fusion sequence.

index: pipeline-assigned index identifying the fusion record or transcript configuration.
fusion_read_support: number of sequencing reads supporting the fusion junction.
fusion_expression: expression value assigned to the fusion event, available when STARfusion output tsv file was provided when running the workflow.
StarFusion: indicator whether the event was detected by STAR-Fusion or not.
fusionMap: indicator whether the event was detected by FusionMap or not.

The `02_mutated_peptide/*_fusion_similarity.csv` file contains the similarity results for the gene-fusion-derived mutated peptides. The output file contains the following columns:
ID: unique identifier assigned to the gene-fusion event, such as g1, g2.
gene_name: name or combined label of the genes involved in the fusion.
fusion_read_support: number of sequencing reads supporting the fusion junction.
fusion_expression: expression value assigned to the fusion event, available when a STAR-Fusion output TSV file was provided when running the workflow.
fusion_amino_acid_sequence: predicted amino acid sequence of the fusion protein from which the n-mer peptides were generated.
Mut_peptide: mutated n-mer peptide generated from the predicted fusion protein sequence.
WT_peptide: wildtype peptide used as the reference for comparison with the mutated peptide.
hamming_similarity: similarity score between the mutated and wildtype peptides based on identical amino acids at corresponding positions.
blosum62_2_similarity: similarity score between the mutated and wildtype peptides calculated using the BLOSUM62 amino acid substitution matrix.

5.2.3 SVs annotation and sequence reconstruction

The `00_variants_summary/*_sv_summary.csv` file contains annotated structural variants (SVs) and the nucleotide and protein sequences generated across their breakpoints. The output file contains following columns:
ID: unique identifier assigned to the SV, such as sv1, sv2.

sv_sorted_coord: standardized representation of the SV coordinates.
chrom1: chromosome containing the first breakpoint.
pos1: genomic position of the first breakpoint.
chrom2: chromosome containing the second breakpoint.
pos2: genomic position of the second breakpoint.
gene1: gene associated with the first transcript.
gene2: gene associated with the second transcript.
sv_gene: gene(s) involved in that SV event.
svlen: estimated length of the SV.
pattern: breakpoint orientation pattern used to reconstruct the SV sequence.
svtype: simplified SV type.
svtype_original: original SV type reported by the upstream variant caller.
somatic_score: quality score of the called somatic SV event.
frameshift_status: predicted reading-frame status of the resulting sequence.
sv_gene_fusion_boolean: Boolean value indicating whether the SV is predicted to produce a gene fusion.
mut_nt_sequence: reconstructed mutant nucleotide sequence spanning the SV breakpoint.
mut_aa_sequence: protein sequence translated from the reconstructed mutant nucleotide sequence.
wt_t1_aa_sequence: wildtype protein sequence corresponding to the first transcript of the SV event.
wt_t2_aa_sequence: wildtype protein sequence corresponding to the second transcript of the SV event.
k: k-mer length used to generate peptide sequences, by default k=9.
mut_kmers: k-mers generated from the mutant protein sequence.
wt_kmers: k-mers generated from the corresponding wildtype protein sequence or sequences.
novel_kmers: number of mutant k-mers that are absent from the corresponding wildtype sequences.
novel_kmers_sequence: list of novel k-mers sequences.

The `02_mutated_peptide/*_sv_similarity.csv` file contains the similarity results for the SV-derived mutated peptides. The output file contains the following columns:
ID: unique identifier assigned to the SV, such as sv1, sv2.

sv_gene: gene or genes involved in the SV event.
novel_kmers_sequence: list of novel k-mer sequences generated from the mutant protein sequence that are absent from the corresponding wildtype protein sequences.
Mut_peptide: novel n-mer peptide generated from the reconstructed mutant protein sequence.
WT_peptide: wildtype peptide used as the reference for comparison with the mutated peptide.
hamming_similarity: similarity score between the mutated and wildtype peptides based on identical amino acids at corresponding positions.
blosum62_2_similarity: similarity score between the mutated and wildtype peptides calculated using the BLOSUM62 amino acid substitution matrix.

5.2.4 HLA typing and HLA loss-of-heterozygosity

The `01_hla_typing/` directory contains patient-level HLA typing output files from T1K [1], including HLA typing from germline WGS/WES data, tumor WGS/WES data, and tumor bulk RNA-seq data
column 1: HLA allele
column 2: quality

The `07_spechla/` directory contains HLA loss-of-heterozygosity (LOH) results generated using SpecHLA [2]. In this workflow, HLA-associated reads are first extracted from the WGS data, full-resolution HLA typing is performed, and tumor allele frequencies are evaluated together with tumor purity and ploidy to identify locus-specific HLA LOH [3].

5.2.5 Peptide-MHC binding affinity, stability, and immunogenicity prediction

The `03_pMHC_prediction/` directory contains peptide-HLA prediction results generated using MHCflurry[4], netMHCpan[5], netMHCstabpan[6], and PRIME2[7]. Predictions are generated separately for peptides derived from SNVs, gene fusions, and SVs.

The following columns are shared by the prediction output:
Patient_ID: identifier of the patient from whom the peptide was derived.
HLA_allele: HLA class I allele for which the peptide was evaluated. The

allele notation may differ between prediction tools, such as HLA-A*29:02 or HLA-A29:02.
Peptide: amino acid sequence of the mutated peptide evaluated by the prediction tool.
Identity: peptide identifier retained from the input FASTA file, such as s1_pep1, g2_pep1, or sv1_pep1.
Prediction_tool: name of the prediction tool used to generate the results.

The following columns are specific by the prediction tools:
MHCflurry:

Affinity: predicted peptide–HLA binding affinity in nanomolar units. Lower values indicate stronger predicted binding.
Affinity_percentile: allele-specific percentile rank of the predicted binding affinity. The value ranges from 0 to 100, and lower values indicate stronger predicted binding relative to the reference peptide distribution.
Processing_score: predicted likelihood that cellular antigen-processing mechanisms will generate the peptide. The score ranges from 0 to 1, and higher values indicate more favourable predicted processing.
Presentation_score: combined peptide-presentation score calculated from the binding-affinity and antigen-processing predictions. The score ranges from 0 to 1, and higher values indicate a greater predicted likelihood of presentation by the HLA molecule.
Presentation_percentile: percentile rank of the presentation score relative to the reference peptide distribution. Lower values indicate stronger predicted presentation.

netMHCpan:

Score_EL: raw eluted-ligand prediction score. Higher values indicate a greater predicted likelihood that the peptide is naturally presented by the HLA molecule.
Rank_EL: percentile rank of the eluted-ligand prediction score compared with a reference set of random natural peptides. Lower values indicate a stronger predicted ligand.
Score_BA: raw binding-affinity prediction score. Higher values indicate stronger predicted peptide–HLA binding.
Rank_BA: percentile rank of the binding-affinity prediction score compared

with a reference set of random natural peptides. Lower values indicate stronger predicted binding.
Affinity: predicted peptide-HLA binding affinity in nanomolar units. Lower values indicate stronger predicted binding.

netMHCstabpan:

Pred: raw peptide–MHC stability prediction score. Higher values indicate greater predicted stability of the peptide–MHC complex.
Thalf(h): predicted half-life of the peptide–MHC complex in hours. Higher values indicate that the peptide is predicted to remain bound to the MHC molecule for a longer period.
%Rank_stab: percentile rank of the predicted stability score compared with a reference set of 200,000 random natural 9-mer peptides. Lower values indicate greater predicted peptide–MHC stability.
1-log50k: transformed binding-affinity prediction score, calculated from the predicted affinity as $1 - \log(\text{Affinity})/\log(50{,}000)$. Higher values indicate stronger predicted binding.
Aff(nM): predicted peptide–HLA binding affinity expressed as an IC50 value in nanomolar units. Lower values indicate stronger predicted binding.
%Rank_aff: percentile rank of the predicted binding-affinity score compared with a reference set of random natural peptides. Lower values indicate stronger predicted binding.
Comb: prediction score combining the peptide–MHC stability and binding-affinity predictions. Higher values indicate a stronger combined prediction.
%Rank_comb: percentile rank derived from the combined stability and binding-affinity score. Lower values indicate a stronger combined prediction.
Ave: average prediction score across the HLA alleles evaluated for the peptide. When only one HLA allele is evaluated, this value corresponds to the score for that allele.
NB: number of evaluated HLA alleles for which the peptide is classified as a binder according to the tool-specific rank threshold.

PRIME2:

%Rank: percentile rank of the PRIME2 immunogenicity score. It represents the percentage of random human-proteome peptides with a PRIME2 score equal to or higher than that of the evaluated peptide. Lower values indicate

a stronger predicted immunogenicity.
Score: raw PRIME2 immunogenicity score for the peptide–HLA combination. The score integrates predicted MHC class I binding and the propensity for T-cell receptor recognition. Higher values indicate stronger predicted immunogenicity.
%RankBinding: peptide-HLA binding percentile rank generated by MixMHCpred, which is used as the binding component of PRIME2. Lower values indicate stronger predicted binding.

Files matching `03_pMHC_prediction/*_with_DAI.csv` additionally compare the predicted binding rank of each mutant peptide with that of its corresponding wildtype peptide.
DAI_*: * denotes denotes the tool-specific prediction rank metric used to calculate the DAI. A value below 1 indicates that the mutant peptide has a lower predicted binding rank than the corresponding wild-type peptide. This column is included only in files with the `_with_DAI.csv` suffix.

For a prediction rank X, the DAI value is calculated as:

$$\mathrm{DAI}(X) = \frac{\text{mutated } X}{\text{wildtype } X}.$$

The 08_immunogenicity/ directory contains predicted immunogenicity scores for peptide-HLA combinations generated using DeepImmuno [8], ICERFIRE [9], and the IEDB MHC class I immunogenicity predictor [10, 11]. Predictions are generated separately for peptides derived from SNVs, gene fusions, and SVs. DeepImmuno uses a convolutional neural network to estimate the confidence that a peptide-HLA combination will elicit a $CD8^+$ T-cell response [8, 12]. ICERFIRE is an ensemble random-forest model that evaluates the optimal HLA-binding ICORE and incorporates predicted antigen presentation, differences between mutated and wildtype peptides, self-similarity, and antigen-expression information [9, 13]. IEDB MHC class I immunogenicity predictor calculates a peptide immunogenicity score from amino-acid properties and their positions within the peptide, while masking residues primarily involved in MHC binding [11, 10].

The following columns are shared by the prediction output:
Patient_ID: patient or sample identifier.

HLA_allele: HLA allele associated with the peptide–HLA prediction.
Peptide: mutated peptide sequence evaluated for immunogenicity.
Identity: pipeline-assigned identifier linking the prediction to its source variant or peptide record, such as g2 or g2_pep1.
Immunogenicity_tool: immunogenicity-prediction tool used to generate the result.

The following columns are specific by the prediction tools:
DeepImmuno:

Immunogenicity_Score: DeepImmuno-CNN immunogenicity score. A higher score indicates greater confidence that the peptide-HLA combination will elicit a T-cell response; however, the score should not be interpreted as a direct measurement of the magnitude of the immune response.

ICERFIRE:

Immunogenicity_Score: ICERFIRE model score for the predicted immunogenicity of the mutated peptide. A higher score indicates stronger predicted immunogenicity.
Immunogenicity_Rank: percentile rank of the ICERFIRE score relative to predictions for a reference set of human-proteome peptides. A lower percentile rank indicates stronger predicted immunogenicity.

IEDB immunogenicity:

Immunogenicity_Score: IEDB MHC class I immunogenicity score. A higher score indicates a greater predicted likelihood that the peptide will elicit a T-cell response.

Prediction results can further be merged with variant-level information, and filtered using specified thresholds defined in the workflow configuration. The filtered files are saved under `04_neoepitopes/` directory. If no peptide passes the configured criteria for a particular variant type and prediction tool, the corresponding output file may contain only the column header or may be empty. These files represent one possible filtering strategy and do not restrict subsequent analyses to the same thresholds. An alternative filtering strategy

can be applied directly through database queries after results insert.

5.2.6 Gene signature scores

The 05_TME/ directory contains patient-level tumor microenvironment (TME) and gene-signature scores calculated from gene expression data. The gene signatures and their corresponding calculation methods are defined in the signature and calculation-method file specified in config.yaml.

`*_gene_signatures.csv` contains the calculated gene signature scores for an individual patient. Each row represents one gene signature defined in the configured signature file. The output file contains the following columns:
score: name of the gene signature or TME feature being evaluated, such as Adhesion molecules, B cell, or CAF.
method: calculation method used to generate the signature value, as defined in the configured signature and calculation-method file. Examples include Simple average and Average zscore.
value: calculated patient-level value for the gene signature. The interpretation and numerical scale of this value depend on the calculation method used.

5.2.7 Additional RNA supporting

The `06_isovar/` directory contains *_isovar.csv, generated by Isovar from the patient-specific variants and tumor RNA-seq alignment file. Isovar uses RNA reads spanning each variant to assemble mutation-supporting coding sequences and translate them into mutant protein subsequences, allowing nearby variants and some alternatively spliced isoforms to be represented in the reconstructed sequence [14].

5.2.8 Tumor ploidy and purity

The `07_amber/` directory contains the native output files generated by AMBER. AMBER identifies heterozygous germline SNP loci in the normal sample, calculates and segments the corresponding tumor B-allele frequencies, and estimates potential sample contamination; the tumor B-allele frequencies are subsequently used by PURPLE for copy-number fitting [15].

The `07_cobalt/` directory contains the native output files generated by COBALT. COBALT calculates GC-corrected and normalized read-depth ratios for ge-

nomic windows in the tumor and normal samples and segments these ratios to identify changes used by PURPLE during copy-number fitting [16].

The `07_purple/` directory contains the native tabular, quality-control, and visualization outputs generated by PURPLE. PURPLE integrates the B-allele frequencies generated by AMBER, the read-depth ratios generated by COBALT, and available variant calls to estimate tumor purity, ploidy, and allele-specific copy number and to identify potential driver events [17].

5.2.9 MS-based target-specific peptide

The 09_diann/ directory contains peptide and protein identification and quantification results generated from data-independent acquisition mass-spectrometry data using DIA-NN version 2.3.2 [18, 19].

The main Parquet report provides the most detailed output for customized downstream analysis. The matrix files provide simplified, analysis-ready representations filtered according to the DIA-NN matrix-generation settings [19].

The `mutation_specific_results/` and `target_specific_results/` directories contain results derived from the DIA-NN main Parquet report, and they are not standard output from DIA-NN. The mutation-specific analysis identifies peptides that occur in mutant proteins but not in their corresponding wildtype proteins, whereas the target-specific analysis identifies peptides specific to the selected human endogenous retrovirus (HERV) target proteins relative to the background protein database.

`all_tagged_peptides_classification.tsv` contains all detected peptide sequences that match at least one tagged mutant or wildtype protein. Each peptide is classified according to whether it occurs in the mutant protein set, the wildtype protein set, or both. The output file contains the following columns:
Stripped.Sequence: detected peptide amino acid sequence without modification annotations.
in_mutant_proteins: Boolean value indicating whether the peptide sequence occurs in at least one mutant protein.
in_wt_proteins: Boolean value indicating whether the peptide sequence occurs in at least one corresponding wildtype protein.
n_mutant_proteins_matched: number of mutant proteins containing the peptide sequence.

n_wt_proteins_matched: number of wildtype proteins containing the peptide sequence.
mutant_protein_ids: semicolon-separated identifiers of the matched mutant proteins.
wt_protein_ids: semicolon-separated identifiers of the matched wildtype proteins.
mutation_specific: Boolean value indicating whether the peptide is specific to the mutant protein set. A value of True indicates that the peptide occurs in at least one mutant protein and is absent from all corresponding wildtype proteins.

`mutation_specific_peptides.tsv` contains peptide-level summaries for sequences classified as mutation-specific. Multiple precursor observations and mass-spectrometry runs are aggregated by the unmodified peptide sequence. The output file contains the following columns:
Stripped.Sequence: peptide amino acid sequence without modification annotations.
n_runs: number of mass-spectrometry runs in which the peptide was detected.
n_precursors: number of distinct precursor records associated with the peptide sequence. Distinct precursors may represent different charge states or modified forms of the same peptide.
Modified.Sequence: peptide sequence including modification annotations, when present.
Protein.Ids: identifiers of all proteins matched to the peptide by DIA-NN.
Protein.Names: names of the matched proteins.
Genes: gene names associated with the matched proteins.
Precursor.Normalised.max: maximum normalized precursor quantity among the observations associated with the peptide.
Precursor.Normalised.mean: arithmetic mean of the normalized precursor quantities associated with the peptide.
Precursor.Normalised.median: median normalized precursor quantity among the observations associated with the peptide.
in_mutant_proteins: Boolean value indicating whether the peptide occurs in at least one mutant protein.
in_wt_proteins: Boolean value indicating whether the peptide occurs in at least one wildtype protein.

n_mutant_proteins_matched: number of mutant proteins containing the peptide sequence.
n_wt_proteins_matched: number of wildtype proteins containing the peptide sequence.
mutant_protein_ids: semicolon-separated identifiers of the matched mutant proteins.
wt_protein_ids: semicolon-separated identifiers of the matched wildtype proteins.
mutation_specific: Boolean value indicating whether the peptide is present in the mutant protein set and absent from the corresponding wildtype protein set.

`target_specific_peptides.tsv` provides the equivalent peptide-level summary for peptides specific to the selected HERV target proteins.

`candidate_mutant_tagged_precursors.tsv` contains DIA-NN precursor identifications assigned to at least one tagged mutant protein.

`mutation_specific_precursors.tsv` contains the subset of precursor identifications whose stripped peptide sequences are classified as mutation-specific. The candidate and mutation-specific precursor files contain the following columns, which retain their meanings from the DIA-NN main report [19]:
Run: name of the mass-spectrometry run in which the precursor was identified.
Precursor.Id: precursor identifier consisting of the modified peptide sequence and precursor charge.
Modified.Sequence: peptide sequence including modification annotations.
Stripped.Sequence: peptide amino acid sequence without modification annotations.
Precursor.Charge: charge state of the identified precursor ion.
Protein.Ids: identifiers of all proteins matched to the precursor in the spectral library or sequence database.
Protein.Group: protein group inferred by DIA-NN for the precursor.
Protein.Names: names of the proteins in the inferred protein group.
Genes: gene names associated with the inferred protein group.
Q.Value: run-specific precursor q-value used to control the false-discovery rate. Lower values indicate greater identification confidence.
Global.Q.Value: precursor q-value calculated across the complete experi-

ment. It measures confidence that the precursor was correctly identified in at least one analyzed run.
Lib.Q.Value: q-value associated with the corresponding spectral-library entry. For a DIA-NN-generated empirical library, this represents the global q-value assigned during library generation.
PEP: run-specific posterior error probability for the precursor identification. Lower values indicate a lower estimated probability that the individual identification is incorrect.
Precursor.Quantity: non-normalized MS2-based precursor quantity.
Precursor.Normalised: normalized MS2-based precursor quantity after correction for systematic technical variation between runs.
RT: observed retention time of the peptide-spectrum match, reported in minutes.
Predicted.RT: retention time predicted from the spectral-library or calibrated retention-time information.
IM: observed ion-mobility value, when ion-mobility information is available.
Predicted.IM: predicted ion-mobility value, when applicable.

`candidate_target_tagged_precursors.tsv` and `target_specific_precursors.tsv` contain the corresponding candidate and HERV-specific precursor identifications and follow the same column definitions.

mutant_proteins_from_description.tsv contains the mutant protein records extracted from the DIA-NN protein-description output. These sequences are used to determine whether detected peptides are specific to the mutant protein set. The output file contains the following columns:
Protein.Id: identifier assigned to the mutant protein sequence.
Protein.Name: protein name associated with the mutant protein record.
Gene: gene name associated with the mutant protein.
Description: protein-description field obtained from the sequence database. NoDescription is reported when no description is available.
Sequence: complete amino acid sequence of the mutant protein.
is_mutant: Boolean value indicating whether the protein record belongs to the mutant protein set.

target_proteins_from_description.tsv contains the equivalent protein records for the selected HERV target proteins used in the target-specific peptide

analysis.

6 Importing results into the database

nepiDx can optionally store processed patient-level results in a database. The database must be initialized before the database-import rules are executed by the main Snakemake workflow.

The scripts required to create and initialize the database are located in the `database\_scripts/` directory.

6.1 Database scripts

```
database_scripts/
|-- db_config.json
|-- run_db_scripts.py
|-- create_db_and_tables.py
|-- insert_datasets_and_signatures.py
‘-- get_neoepitope_scores_from_DB.py
```

`database_scripts/db_config.json`

Configuration file containing the settings required to create and initialize the database.

`database_scripts/run_db_scripts.py`

Main database setup script. This script executes the required database-creation and initialization steps.

`database_scripts/create_db_and_tables.py`

Creates the database and its required tables.

`database_scripts/insert_datasets_and_signatures.py`

Inserts the patient and sample definitions together with the available gene signatures.

`database_scripts/get_neoepitope_scores_from_DB.py`

Calculates patient-level neoepitope scores from the database. Usage instructions are provided in the script.

6.2 Creating the database

To create and initialize the database:

- Change to the `database_scripts/` directory.
- Edit `db_config.json` and specify all required values, including the path at which the database will be created.
- Run the database setup script shown below.

```
python run_db_scripts.py
```

The setup script creates the database, initializes the required tables, and inserts the configured patient, sample, and gene-signature definitions.

6.3 Inserting pipeline results

After the database has been created, the main nepiDx workflow can insert patient results during the Snakemake run.

The database path configured for the pipeline must be identical to the path specified in `database_scripts/db_config.json`. In the main pipeline configuration file, this path is defined by the `patient_database_path` parameter.

After setting this parameter, run the main Snakemake workflow as described in the preceding sections.

A patient is inserted only when all required intermediate and final result files are available. Patients with incomplete results are skipped.

6.4 Database import requirements

- The database must be created before the database-import rules are executed.
- The value of `patient_database_path` in `config.yaml` must match the database path specified in `db_config.json`.

- Only patients for which all required results are available are inserted.
- A patient is skipped when one or more required intermediate or final output files are missing.

References

[1] Li Song, Gali Bai, X Shirley Liu, Bo Li, and Heng Li. Efficient and accurate kir and hla genotyping with massively parallel sequencing data. *Genome Research*, 33(6):923–931, 2023.

[2] Shuai Wang, Mengyao Wang, Lingxi Chen, Guangze Pan, Yanfei Wang, and Shuai Cheng Li. SpecHLA enables full-resolution HLA typing from sequencing data. *Cell Reports Methods*, 3(9):100589, 2023.

[3] DeepOmics Lab. SpecHLA: Full-resolution HLA typing from sequencing data. `https://github.com/deepomicslab/SpecHLA`. Accessed: 16 August 2026.

[4] Timothy J O'Donnell, Alex Rubinsteyn, and Uri Laserson. Mhcflurry 2.0: improved pan-allele prediction of mhc class i-presented peptides by incorporating antigen processing. *Cell systems*, 11(1):42–48, 2020.

[5] Birkir Reynisson, Bruno Alvarez, Sinu Paul, Bjoern Peters, and Morten Nielsen. Netmhcpan-4.1 and netmhciipan-4.0: improved predictions of mhc antigen presentation by concurrent motif deconvolution and integration of ms mhc eluted ligand data. *Nucleic acids research*, 48(W1):W449–W454, 2020.

[6] Michael Rasmussen, Emilio Fenoy, Mikkel Harndahl, Anne Bregnballe Kristensen, Ida Kallehauge Nielsen, Morten Nielsen, and Søren Buus. Pan-specific prediction of peptide–mhc class i complex stability, a correlate of t cell immunogenicity. *The Journal of Immunology*, 197(4):1517–1524, 2016.

[7] David Gfeller, Julien Schmidt, Giancarlo Croce, Philippe Guillaume, Sara Bobisse, Raphael Genolet, Lise Queiroz, Julien Cesbron, Julien Racle, and Alexandre Harari. Improved predictions of antigen presentation and tcr recognition with mixmhcpred2. 2 and prime2. 0 reveal potent sars-cov-2 cd8+ t-cell epitopes. *Cell Systems*, 14(1):72–83, 2023.

[8] Guangyuan Li, Balaji Iyer, V. B. Surya Prasath, Yizhao Ni, and Nathan Salomonis. DeepImmuno: Deep learning-empowered prediction and generation of immunogenic peptides for t-cell immunity. *Briefings in Bioinformatics*, 22(6):bbab160, 2021.

[9] Yat-tsai Richie Wan, Zeynep Koşaloğlu-Yalçın, Bjoern Peters, and Morten Nielsen. A large-scale study of peptide features defining immunogenicity of cancer neo-epitopes. *NAR Cancer*, 6(1):zcae002, 2024.

[10] Immune Epitope Database and Analysis Resource. MHC class i immunogenicity prediction tool. `https://tools.iedb.org/immunogenicity/`. Accessed: 16 August 2026.

[11] Jorg J. A. Calis, Matt Maybeno, Jason A. Greenbaum, Daniela Weiskopf, Aruna D. De Silva, Alessandro Sette, Can Keşmir, and Bjoern Peters. Properties of MHC class i presented peptides that enhance immunogenicity. *PLoS Computational Biology*, 9(10):e1003266, 2013.

[12] Guangyuan Li. DeepImmuno: Deep-learning empowered prediction and generation of immunogenic epitopes. `https://github.com/frankligy/deepimmuno`. Accessed: 16 August 2026.

[13] Yat-tsai Richie Wan. ICERFIRE: Icore-based ensemble random forest for neo-epitope immunogenicity prediction. `https://github.com/richieYT-wan/ICERFIRE`. Accessed: 16 August 2026.

[14] OpenVax. Isovar: Assembly of RNA reads to determine the effect of a cancer mutation on protein sequence. `https://github.com/openvax/isovar`. Accessed: 16 August 2026.

[15] Hartwig Medical Foundation. AMBER: Generation of tumor b-allele frequencies for copy-number fitting. `https://github.com/hartwigmedical/hmftools/tree/master/amber`. Accessed: 16 August 2026.

[16] Hartwig Medical Foundation. COBALT: Read-depth ratio calculation for copy-number fitting. `https://github.com/hartwigmedical/hmftools/tree/master/cobalt`. Accessed: 16 August 2026.

[17] Hartwig Medical Foundation. PURPLE: Purity, ploidy, copy-number, and driver analysis. `https://github.com/hartwigmedical/hmftools/tree/master/purple`. Accessed: 16 August 2026.

[18] Vadim Demichev, Christoph B. Messner, Spyros I. Vernardis, Kathryn S. Lilley, and Markus Ralser. DIA-NN: Neural networks and interference correction enable deep proteome coverage in high throughput. *Nature Methods*, 17(1):41–44, 2020.

[19] Vadim Demichev. DIA-NN: A universal automated software suite for DIA proteomics data analysis. https://github.com/vdemichev/DiaNN. Version 2.3.2; accessed: 16 August 2026.